

Tier 2 Stateful Repo Engineering Task Guide

General Multi-Step Harbor Packaging, Stage Checks, Anti-Cheating, Judges, Rewards, and Training Design

1. What a Tier 2 Task Is

A Tier 2 stateful repo-engineering task is one engineering assignment split into ordered stages. Each stage starts with a fresh agent context, but the repository, services, and explicit handoff files persist. The benchmark therefore measures whether useful engineering work survives context boundaries and whether the next agent call can diagnose, implement, respond to feedback, recover, and harden the same system.

Property	Tier 2 requirement	Why it matters
Repository	Complete production-scale worktree at an exact pre-change commit	Preserves real navigation, build, test, and integration complexity
State	Repository and services persist across stages	Measures durable work rather than independent short answers
Context	Fresh agent context per stage	Tests handoffs and reconstruction of engineering state
Feedback	Later prompts reveal review, CI, production, or fault evidence	Measures response to new information
Evaluation	Stage gates plus a cumulative final deterministic closure	Allows recovery without weakening final correctness
Target headroom	Roughly 5-20% strict frontier solve rate after calibration	Leaves room above current single-session systems

2. How It Differs from Tier 1 and Senior SWE-Bench

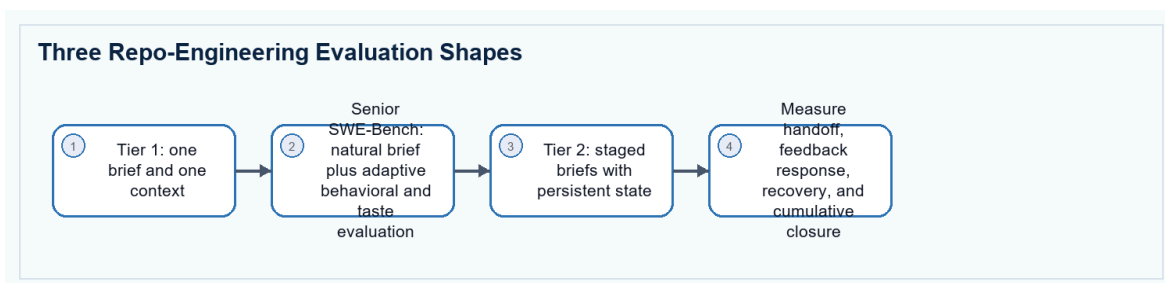


Figure 1. Tier 2 adds state-over-time measurement; it does not discard the strong behavioral and quality ideas already present in Senior SWE-Bench.

Design	Task and context	Reward stack	Primary measurement
Tier 1 in this project	One natural issue brief, one agent context, one final repository	Fixed executable checks plus optional quality judges	Issue-to-patch execution in one uninterrupted run
Senior SWE-Bench v2026.06	Natural, under-specified single task in a real repository	Behavioral verifiers, adaptive validation agent, task rubric, taste judge	Senior-level correctness and taste for one task submission

Design	Task and context	Reward stack	Primary measurement
Tier 2 in this guide	Ordered fresh stage briefs over one persistent repository and service state	Per-stage executable gates, cumulative final closure, stage-aware judges	Durable work, handoff use, response to feedback, recovery, and hardening over time

Senior SWE-Bench is already a strong long-horizon single-task benchmark. Its June 2026 design uses recent real PRs, natural briefs, behavioral verifiers, an adaptive validation agent, task rubrics, taste scoring, and repeated Oracle/no-op quality control. The current site lists 100 tasks, split into 50 public and 50 private. As of July 14, 2026, its reported top tasteful solve rate is 29.1%, so it already has substantial model headroom.

The clear difference

Senior SWE-Bench evaluates whether one submission solves a difficult senior-level task correctly and tastefully. Tier 2 evaluates the same kind of real repository work while the task evolves across fresh contexts: first diagnosis, then implementation, then review/CI feedback, then operational hardening. The extra signal is temporal and stateful, not merely a larger patch.

What Tier 2 should borrow

- Natural issue-style briefs instead of a full patch specification.
- Behavioral fail-to-pass and pass-to-pass checks that accept multiple valid implementations.
- Quality judges grounded in repository practices and bounded evidence.
- Repeated Oracle, no-op, guided, near-miss, and frontier calibration.
- Strict internet controls and trajectory review for reward hacking.

What Tier 2 adds

- Fresh context at every stage while code and services persist.
- A typed handoff artifact that later stages must consume.
- New review, CI, production, or fault evidence revealed only after earlier work.
- Promotion thresholds that permit a bounded near miss to continue and test recovery.
- Per-stage reward events and snapshots for temporal credit assignment.
- A cumulative final verifier that closes earlier gaps before strict success.

3. End-to-End Stateful Flow

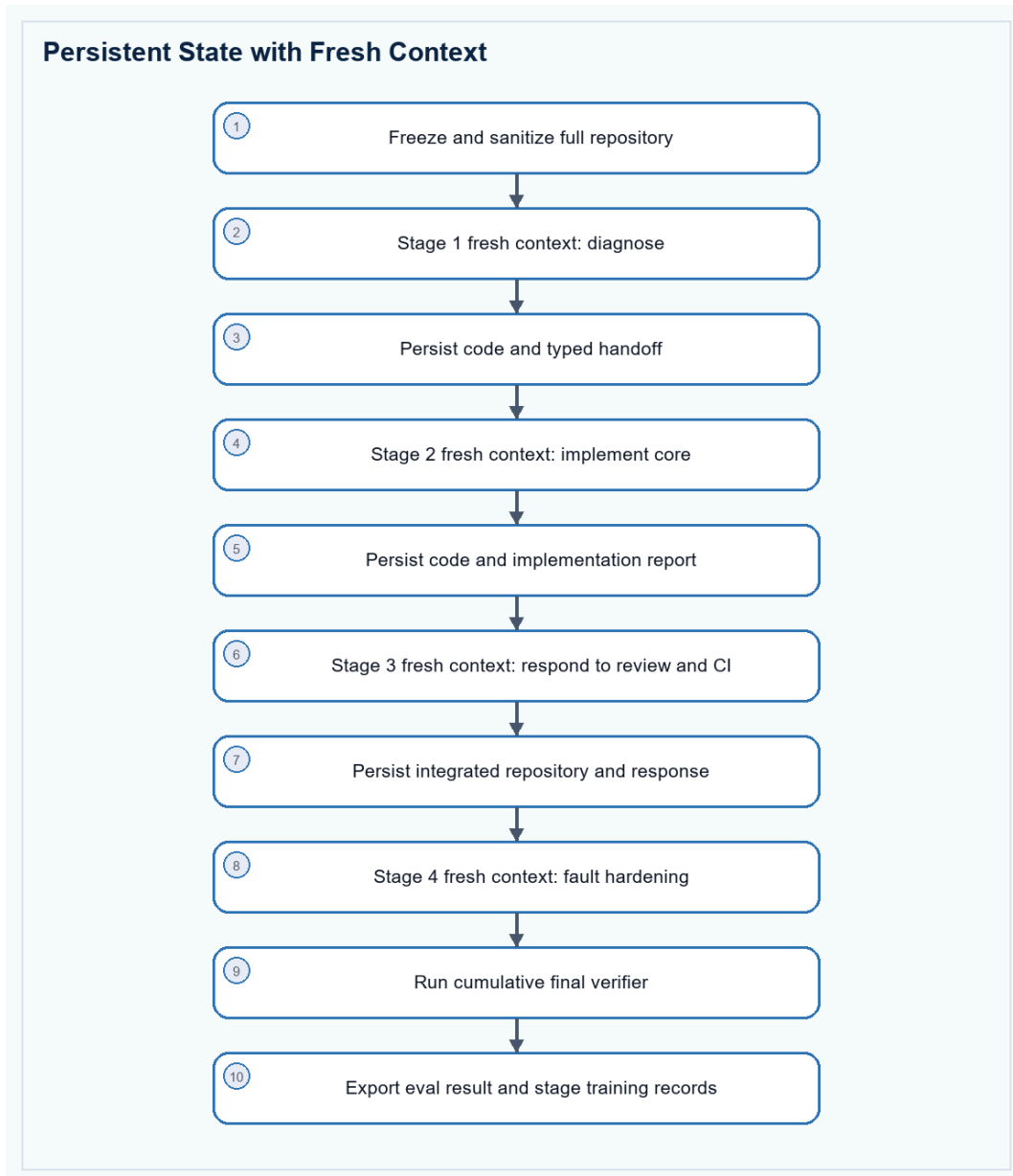


Figure 2. Context resets between stages, while the repository and explicit engineering artifacts remain durable.

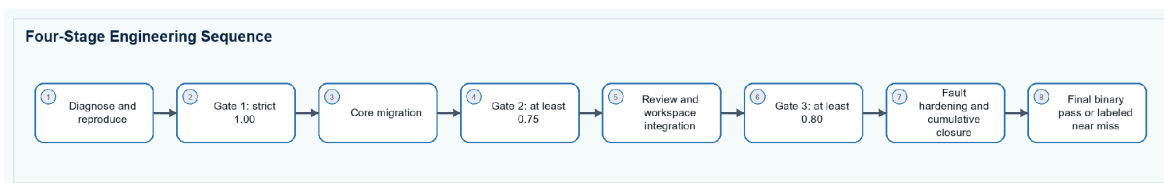


Figure 3. Early gates control promotion; only the final all-pass result counts as a strict benchmark solve.

Use `multi_step_reward_strategy = "final"` when the final verifier rechecks the complete repository. If an early `min_reward` gate aborts, Harbor returns that aborted stage's result. Consumers must therefore

use ``binary_reward``, completion state, and stage identity together; a partial early reward is training signal, not a solved task.

4. Recommended Harbor Package

Generic Tier 2 Harbor Package

```
<task-name>/
|-- task.toml                # shared config, ordered steps, gates, limits
|-- README.md                # author-facing runbook; not an agent prompt
|-- environment/
|   +- Dockerfile            # sanitized full repo and pinned verifier runtime
|-- tests/                   # shared protected verifier helpers
|   |-- stage_test.sh        # runs deterministic stage check, then optional judge
|   |-- stage_check.py       # executable checks, weights, reward events, manifests
|   |-- stage_judge_inputs.py # stages bounded evidence for RewardKit
|   |-- run_optional_judges.sh # stage-aware Gemini judge runner
|   |-- test_reward_contract.py # discoverable package and judge-selection tests
|   |-- hidden_behavior.rs    # verifier-owned hidden behavior cases
|   |-- judge_prompt.md       # shared bounded judge instructions
|   |-- process/judge.toml    # reasoning and evidence quality
|   |-- validity/judge.toml   # leakage, shortcuts, failure classification
|   |-- task_quality/judge.toml # author-facing prompt/verifier alignment
|   +- merge_readiness/judge.toml # final code quality; cannot rescue failure
|-- steps/
|   |-- 01-diagnose/
|   |   |-- instruction.md    # fresh Stage 1 brief
|   |   |-- tests/test.sh    # calls /tests/stage_test.sh diagnose
|   |   +- solution/solve.sh  # author-only Stage 1 Oracle
|   |-- 02-core-migration/
|   |   |-- instruction.md
|   |   |-- tests/test.sh    # calls /tests/stage_test.sh core
|   |   +- solution/         # solve.sh plus private patch/provenance
|   |-- 03-review-integration/
|   |   |-- instruction.md
|   |   |-- tests/test.sh    # calls /tests/stage_test.sh review
|   |   +- solution/
|   +- 04-hardening-final/
|       |-- instruction.md
|       |-- tests/test.sh    # calls /tests/stage_test.sh harden
|       +- solution/solve.sh
+- tools/
    +- run_inline_judged_terminus.sh # one command for agent plus inline judges
```

Figure 4. A multi-step task replaces the root prompt and solution with ordered stage directories and keeps protected shared verifier helpers at the task root.

```
<task-name>/
|-- task.toml                # shared config, ordered steps, gates, limits
|-- README.md                # author-facing runbook; not an agent prompt
|-- environment/
|   +- Dockerfile            # sanitized full repo and pinned verifier runtime
|-- tests/                   # shared protected verifier helpers
|   |-- stage_test.sh        # runs deterministic stage check, then optional judge
|   |-- stage_check.py       # executable checks, weights, reward events, manifests
|   |-- stage_judge_inputs.py # stages bounded evidence for RewardKit
|   |-- run_optional_judges.sh # stage-aware Gemini judge runner
|   |-- test_reward_contract.py # discoverable package and judge-selection tests
|   |-- hidden_behavior.rs    # verifier-owned hidden behavior cases
|   |-- judge_prompt.md       # shared bounded judge instructions
|   |-- process/judge.toml    # reasoning and evidence quality
|   |-- validity/judge.toml   # leakage, shortcuts, failure classification
|   |-- task_quality/judge.toml # author-facing prompt/verifier alignment
|   +- merge_readiness/judge.toml # final code quality; cannot rescue failure
|-- steps/
|   |-- 01-diagnose/
|   |   |-- instruction.md    # fresh Stage 1 brief
|   |   |-- tests/test.sh    # calls /tests/stage_test.sh diagnose
|   |   +- solution/solve.sh  # author-only Stage 1 Oracle
|   |-- 02-core-migration/
|   |   |-- instruction.md
|   |   |-- tests/test.sh    # calls /tests/stage_test.sh core
|   |   +- solution/         # solve.sh plus private patch/provenance
|   |-- 03-review-integration/
```

```

| | | -- instruction.md
| | | -- tests/test.sh          # calls /tests/stage_test.sh review
| | | ++ solution/
| | | ++ 04-hardening-final/
| | | | -- instruction.md
| | | | -- tests/test.sh        # calls /tests/stage_test.sh harden
| | | ++ solution/solve.sh
|-- tools/
++ run_inline_judged_terminus.sh # one command for agent plus inline judges

```

Important Harbor layout rule

In Harbor 0.8, a true multi-step task uses `steps/<name>/instruction.md`, `steps/<name>/tests/test.sh`, and `steps/<name>/solution/solve.sh`. Do not add a root `instruction.md` merely to silence a checker written for single-step tasks; an older Harbor runner can then misclassify the package and look for a root solution. Use Harbor 0.8 or newer for authoring and execution.

File responsibilities

Path	Purpose	Required content	Protected from agent
task.toml	Harness contract	Ordered steps, promotion gates, final strategy, timeouts, resources, metadata	No secrets or public provenance
environment/Dockerfile	Reproducible state	Sanitized full repo, pinned toolchain, cached dependencies, verifier runtime	No remote history, solutions, or tests
steps/*/instruction.md	Fresh stage brief	Current evidence, current milestone, preserved behavior, durable artifact	No hidden test map or patch recipe
steps/*/solution	Stage Oracle	Repeatable author solve and private provenance	Always
steps/*/tests/test.sh	Stage endpoint	One short call to the shared stage verifier	Copied only at verification
tests/stage_check.py	Deterministic truth	Executable checks, weights, strict gate, reward events, manifests	Always
tests/*/judge.toml	Optional semantic review	Bounded evidence, explicit rubric, model ID	Always
tools/run_inline_judged_terminus.sh	Operator wrapper	Version check, unique job, key injection, bounded repeats	Author/operator only

5. Configuration, Runtime, and Network Boundary

```

schema_version = "1.3"
multi_step_reward_strategy = "final"

[task]
name = "<benchmark>/<task-slug>"
description = "Stateful full-repository work across diagnosis, implementation, review, and hardening."
keywords = ["repo-engineering", "stateful", "multi-step", "tier-2", "<language>"]

[metadata]
benchmark_type = "swe-bench"
category = "software-engineering"
language = "<language>"
tier = "stateful-long-horizon-engineering"
repository = "<owner/repository>"
base_commit = "<exact-pre-change-commit>"
expert_time_estimate_hours = 24.0

[environment]
build_timeout_sec = 7200.0
allow_internet = <false-or-documented-shared-judge-exception>
cpus = 8
memory_mb = 16384
storage_mb = 65536
env = { PATH = "/opt/verifier-venv/bin:<toolchain>:/usr/local/bin:/usr/bin:/bin" }

```

Tier 2 Stateful Repo Engineering Task Guide

```
[[steps]]
name = "01-diagnose"
min_reward = 1.0
artifacts = ["/app/output/handoff.json", "/app/output/01-diagnose"]
[steps.agent]
timeout_sec = 7200.0
[steps.verifier]
timeout_sec = 1200.0

[[steps]]
name = "02-core-migration"
min_reward = 0.75
artifacts = ["/app/output/handoff.json", "/app/output/02-core"]
[steps.agent]
timeout_sec = 14400.0
[steps.verifier]
timeout_sec = 3600.0

[[steps]]
name = "03-review-integration"
min_reward = 0.80
artifacts = ["/app/output/03-review"]
[steps.agent]
timeout_sec = 10800.0
[steps.verifier]
timeout_sec = 3600.0

[[steps]]
name = "04-hardening-final"
artifacts = ["/app/output/04-hardening"]
[steps.agent]
timeout_sec = 10800.0
[steps.verifier]
timeout_sec = 5400.0
```

Generic Dockerfile pattern

```
FROM python:3.12-bookworm AS verifier-python
ARG PYTEST_VERSION=8.3.5
ARG REWARDKIT_VERSION=0.1.7
RUN python -m venv /opt/verifier-venv \
    && /opt/verifier-venv/bin/pip install --no-cache-dir \
        "pytest==${PYTEST_VERSION}" \
        "harbor-rewardkit[all]==${REWARDKIT_VERSION}"

FROM <pinned-language-toolchain>
ARG BASE_COMMIT=<exact-pre-change-commit>
COPY --from=verifier-python /opt/verifier-venv /opt/verifier-venv
ENV PATH=/opt/verifier-venv/bin:<language-toolchain>:/usr/local/bin:/usr/bin:/bin

# Network is allowed while building. Export only the exact base tree so the
# solving agent receives no origin, remote refs, reflog, future commits, PR ID,
# author solution, or verifier files.
RUN git clone --no-tags <repository-url> /tmp/source \
    && git -C /tmp/source checkout --detach "${BASE_COMMIT}" \
    && mkdir -p /testbed \
    && git -C /tmp/source archive "${BASE_COMMIT}" | tar -x -C /testbed \
    && rm -rf /tmp/source

WORKDIR /testbed
RUN <fetch-locked-dependencies-and-prebuild-expensive-targets>
RUN git init && git add -A \
    && git -c user.name=benchmark -c user.email=benchmark@invalid \
        commit -m "sanitized benchmark base"
RUN mkdir -p /app/output /logs/agent /logs/verifier
CMD ["sleep", "infinity"]
```

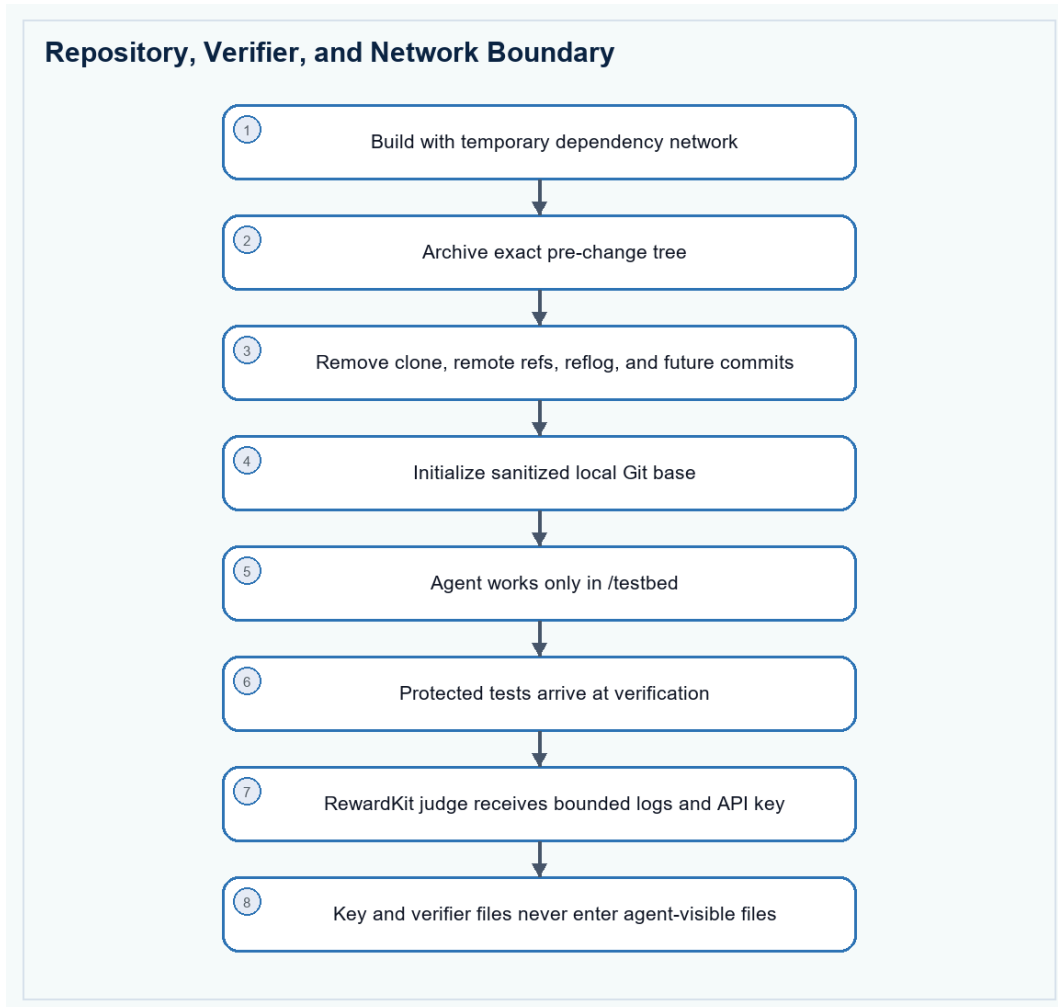


Figure 5. Repository provenance and verifier material stay outside the solving surface; judge credentials exist only in the verifier process.

Current sample network compromise

The worked rustls sample sets `environment.allow_internet = true` because its shared verifier must call Gemini inline and the local provider cannot change network policy after container start. The prompt and process-integrity audit forbid agent network use, but this is weaker than technical isolation. For a production benchmark, prefer a separate verifier environment with a public or strict provider allowlist and keep the agent baseline at no-network or a dependency-only allowlist.

- Pin the language toolchain, locked dependency set, Python 3.12 verifier environment, pytest, and RewardKit.
- Prebuild expensive repository targets so stage time measures engineering rather than first-use downloads.
- Use a sanitized archive at the exact base commit, then initialize a new local Git repository with no origin.
- Never place Gemini keys in task files, images, Markdown, trajectories, or source control.
- Bound judge repeats to 1-3 and each provider call to a hard timeout.

History sanitization: remove source Git memory



Figure 5A. Exporting the exact tree into a new repository removes source history without rewriting or deleting the author's checkout.

```

# Run only while building a disposable benchmark image.
ARG BASE_COMMIT=<exact-pre-change-commit>
RUN set -eux; \
  git clone --no-tags <repository-url> /tmp/source; \
  git -C /tmp/source checkout --detach "$BASE_COMMIT"; \
  mkdir -p /testbed; \
  git -C /tmp/source archive "$BASE_COMMIT" | tar -x -C /testbed; \
  rm -rf -- /tmp/source; \
  cd /testbed; \
  git init; \
  git config user.name benchmark; \
  git config user.email benchmark@invalid; \
  git add -A; \
  git commit -m "sanitized benchmark base"

# Build-time assertions: one synthetic commit, no source remotes/tags/history.
RUN set -eux; \
  cd /testbed; \
  test "$(git rev-list --all --count)" -eq 1; \
  test -z "$(git remote)"; \
  test -z "$(git tag --list)"; \
  test ! -e .git/refs/remotes; \

```

```
test ! -e .git/logs/refs/remotes; \
test "$(git reflog --all --format=%H | sort -u)" = "$(git rev-parse HEAD)"
```

Do not destroy the source checkout

The safest method is export-and-reinitialize, not history rewriting. Run it only in a disposable Docker build directory. Never run `rm -rf .git`, `git filter-repo`, `git reset --hard`, or `reflog expiry` in the maintainer's working repository. If a copied `/testbed/.git` must be removed, first assert `realpath /testbed` is exactly `/testbed`, then delete only `/testbed/.git` inside the image build and immediately create the synthetic base repository.

Leak surface	What to remove or block	Build-time proof	Run-time proof
Git provenance	Origin, remote refs, tags, source objects, source reflogs, branch names, PR refs	Exactly one synthetic commit; no remotes/tags/remote logs	Trajectory rejects history/remote commands
Future solution clues	Future commits, patch files, changelog entries, generated snapshots, source PR number	Denylist scan of the exported tree and image	Hidden variants make recalled public patch insufficient
Author material	<code>/solution/</code> , golden patch, provenance notes, Oracle logs	Never copy into agent image or shared mount	Artifact inventory and path audit
Verifier material	Protected tests, rubrics, reward scripts, expected values	Mount only for verifier execution	Reject reads of verifier paths and reward files
Credentials	Judge keys, proxy tokens, package credentials	No secret in image history or task files	Inject only into verifier process; redact logs
Network	Source hosts, search, arbitrary package installation	Dependencies fetched and cached during build	Block agent network or enforce a dependency-only allowlist
Host escape	Docker socket, host mounts, <code>/proc</code> secrets, sibling jobs	Minimal mounts and non-privileged container	Audit commands, symlinks, hard links, and artifact paths
Reward spoofing	Agent-written reward, fake logs, canned markers	Reward directory is verifier-owned	Verifier reruns behavior and records raw outputs

Anti-cheating controls are part of validity

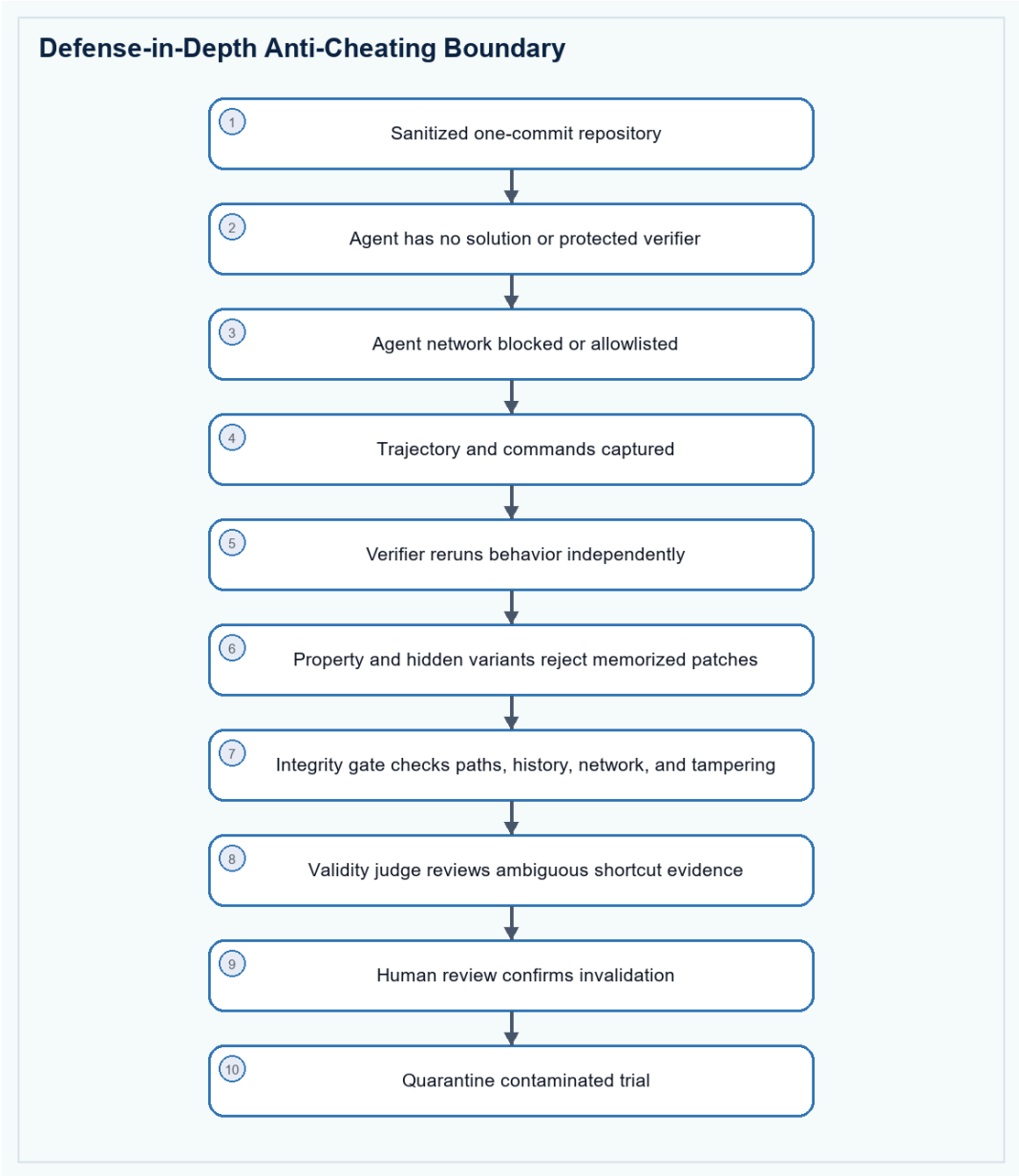


Figure 5B. Anti-cheating uses environment isolation, executable checks, process auditing, semantic review, and quarantine rather than one fragile detector.

Threat	Deterministic control	Judge or review control	Failure handling
Public-patch recall	Novel hidden inputs, configurations, fault points, and cross-component contracts	Validity review checks suspicious unexplained exactness	Label suspected contamination; do not fail solely on style similarity
Verifier discovery	Protected mount, forbidden-path audit, verifier checksums	Review trajectory for probing or indirect disclosure	Set 'valid_trial = 0' on confirmed access
Network retrieval	Blocked network or strict allowlist plus command audit	Review unexplained external facts and downloader attempts	Invalidate confirmed retrieval; preserve evidence
Fake evidence	Verifier reruns reproducer, tests, benchmark, migration, and rollback	Judge compares claims with raw logs and patch	Fail the named evidence gate

Tier 2 Stateful Repo Engineering Task Guide

Threat	Deterministic control	Judge or review control	Failure handling
Test weakening	Checksum protected tests; detect skipped, deleted, filtered, or relaxed project tests	Merge-readiness judge reviews suspicious test edits	Fail regression/integrity gate
Compatibility shortcut	Independent external consumer and source/runtime invariants	Process/validity judge examines duplicate paths and deferred debt	Fail architecture or integration gate
Judge manipulation	Judge cannot modify deterministic rewards; bounded evidence directory	Repeat calls and record disagreement	Mark judge unavailable/uncertain, preserve binary truth
Resource abuse	CPU, memory, storage, process, and wall-clock limits	Review runaway generation or irrelevant work	Classify timeout/resource failure separately

6. Prompt and Handoff Contract

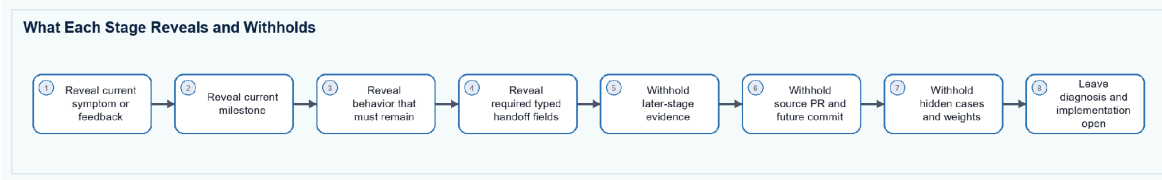


Figure 6. Stage prompts provide enough operational context for fair work while preserving future uncertainty and solution freedom.

```

# Stage <N>: <natural engineering milestone>

The complete repository is at `~/testbed`. Continue from the persistent worktree
and read only the handoff artifacts explicitly named below.

## Current evidence or feedback

Describe the symptom, new CI result, reviewer request, production observation,
or fault evidence revealed at this stage. Do not reveal the source PR, golden
patch, hidden test names, exact implementation, or reward weights.

## Work for this stage

- State the observable behavior or engineering boundary to achieve now.
- State what must remain unchanged.
- State which downstream work is intentionally deferred to a later stage.
- Require executable verification, not a prose-only claim.

## Durable handoff

Write `~/app/output/<stage>/<artifact>.json` with explicit typed fields. Use
structured fields for decisions that a strict gate must check; do not search
free-form prose for one exact phrase.
  
```

Use typed fields for strict handoffs

A handoff can contain free-form explanation, but any decision used by a strict deterministic gate should have a typed field. This avoids the earlier failure mode where a technically correct explanation lost credit for not using one exact phrase.

```

{
  "encrypted_input_owner": "caller",
  "input_buffer_type": "VecInput",
  "caller_populates_input": true,
  "connection_keeps_second_encrypted_buffer": false
}
  
```

Prompt part	What to include	What to avoid
Current evidence	User symptom, compiler result, CI failure, review comment, or fault observation	Golden diff or hidden-test text
Current scope	What this stage must change and what is deliberately deferred	Demanding downstream work before its stage
Preserved behavior	Compatibility, state, security, data, protocol, and performance invariants	Vague 'do not break anything' alone
Artifact	Explicit path, schema, fields, value types, and executable evidence	Wording-sensitive prose keyword checks
Freedom	Behavioral outcome and architecture only when load-bearing	Exact files, symbols, or patch recipe unless truly public API

7. Stage-by-Stage Worked Design

The four stages below are a reusable engineering pattern, not a rustls-only workflow. Replace the language-specific commands and behavior names with the repository's real compiler, services, contracts, migrations, and operational risks. Keep the authority boundaries unchanged: executable behavior decides correctness, integrity decides trial validity, and judges add bounded process and quality signals.

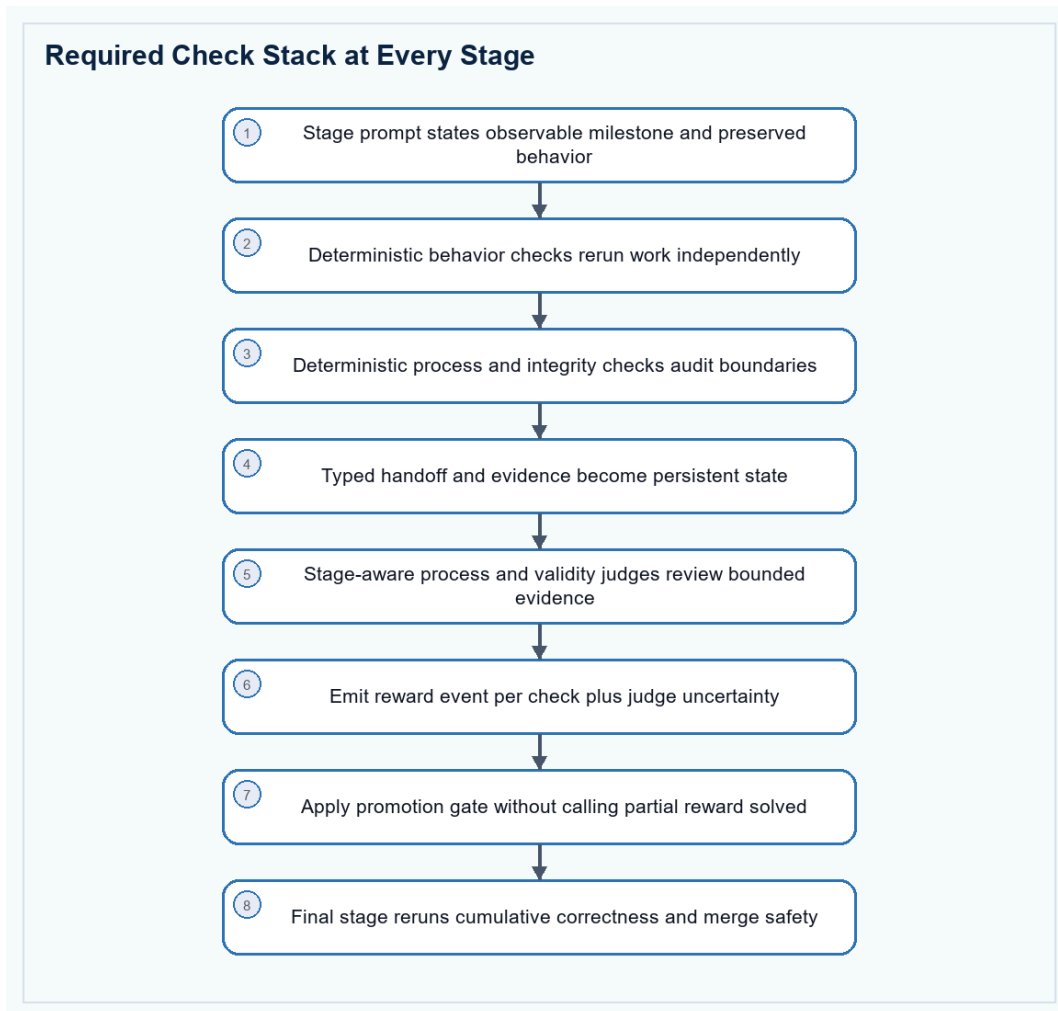


Figure 6A. Every stage combines executable truth, process integrity, durable evidence, semantic review, and an explicit promotion decision.

Stage	Required deterministic truth	Required process and anti-cheat checks	Required non-deterministic review
1. Diagnose	Reproduce the real base failure; prove base state; validate typed diagnosis and executable evidence; confirm production change was not applied	No source history/network/verifier access; no production edit; reproducer is executable, offline where required, and cannot swallow failure	Process: evidence before conclusion. Validity: no fabricated logs or leaked solution. Task quality: every strict contract was disclosed
2. Core change	Fail-to-pass behavior; focused pass-to-pass; external/public contract; relevant feature/configuration matrix; handoff consumed	No compatibility shim that evades the requested boundary; no protected-test edits; exact changed-file and command audit; deferred work remains honestly reported	Process: handoff use, localization, verification, uncertainty. Validity: no reward gaming, hidden second path, or unsupported completion claim

Stage	Required deterministic truth	Required process and anti-cheat checks	Required non-deterministic review
3. Review/integration	Reviewer/CI case fixed; downstream consumers/services pass; old path removed where required; regression and migration/compatibility checks pass	Feedback cannot be pre-revealed; project tests cannot be weakened; integration evidence is verifier-generated; persistent artifacts remain untampered	Process: response to feedback and cross-component reasoning. Validity: no selective test filtering or concealed debt
4. Harden/final	Hidden/property variants; fixed faults; performance budget; rollback/recovery; broad pass-to-pass; cumulative final contract	Recheck all earlier boundaries; audit final patch, history, network, tests, artifacts, and trajectory; verify no late shortcut restored an obsolete path	Process and validity plus task quality. Merge readiness only after deterministic pass: minimality, conventions, architecture, tests, and operability

Artifacts and promotion evidence by stage

Stage	Persistent artifacts	Promotion evidence	Typical failure label
Diagnosis	Reproducer, raw log, typed handoff, affected boundaries and risks	All diagnosis and integrity checks pass	unreproduced, wrong-base-state, ambiguous-handoff, process-boundary
Core	Patch, focused test outputs, implementation report, known downstream work	Critical behavior and public contract pass; bounded partial threshold may permit recovery	core-behavior, public-contract, feature-matrix, shortcut
Review	Updated patch, CI/review response, downstream outputs, migration evidence	Feedback case and minimum integration threshold pass	feedback-unresolved, regression, stale-callsite, test-weakening
Hardening	Fault matrix, benchmark, rollback/recovery check, final report	Every cumulative deterministic and integrity gate passes	fault, performance, rollback, cumulative-regression, contamination

Prompt-verifier fairness rule

If the verifier requires a particular command option, file path, schema field, timeout behavior, or artifact format, the stage prompt must disclose it unless the environment enforces it behaviorally. Return exact machine-readable failure reasons, such as ``missing_contract_requirements: ["uses_explicit_offline_mode"]``, and include them in judge context. A judge must not invent a different cause when deterministic evidence already names the failure.

Stage 1: Diagnosis and executable reproduction

Ask the agent to inspect the current system without applying the production migration. It must produce an executable reproducer, capture real compiler or runtime evidence, and write a typed handoff describing the intended ownership boundary and risks.

Deterministic check	Weight	Pass	Fail
executable_reproduction	0.40	Offline probe runs, observes the real API error, and reports the expected mismatch	Prose only, fabricated log, swallowed error, or wrong call shape
verified_handoff	0.25	Invariant, evidence, affected boundaries, risks, and next checks are populated	Missing or empty technical fields
architecture_specificity	0.20	Typed fields unambiguously select caller-owned Vecinput and no second buffer	Wrong values or natural-language phrase matching
base_api_observed	0.15	Base repository still has the old API; no premature implementation	Production migration started in diagnosis

Stage 1 gate

All checks and integrity must pass. ``min_reward = 1.0``; any lower score stops the trial. No-op receives 0.15 only for observing the untouched base API and therefore cannot advance.

Stage 2: Core migration

Ask the next fresh context to consume the handoff and implement only the core ownership/API boundary. Downstream workspace callsites are intentionally deferred. The verifier must not fail this stage for code the prompt explicitly reserves for Stage 3.

Deterministic check	Weight	Pass	Fail
core_build	0.15	Core library compiles	Core compile or type failure
external_api_contract	0.25	Independent consumer compiles against caller-owned input API	Only internal code changed or public shape is wrong
core_feature_matrix	0.20	No-default and all-feature core builds pass	Feature-specific breakage
ownership_boundary	0.25	Caller-owned input, EOF/close hooks, no second deframer buffer	Compatibility shim or duplicate ownership
handoff_consumed	0.10	Typed Stage 1 handoff persists and remains valid	Handoff discarded or corrupted
implementation_report	0.05	Structured core decisions, tests, and deferred failures	Missing or prose-only report

Stage 2 gate

Strict stage completion is 1.0, but ``min_reward = 0.75`` permits a bounded partial implementation to continue so later stages can measure recovery. ``binary_reward`` stays 0 until all current checks pass. Broad workspace regression checks belong in Stage 3 and the cumulative final gate.

Stage 3: Review response and repository integration

Reveal reviewer and CI feedback only now. Require migration of real downstream callers, removal of the old competing call shape, reuse of long-lived buffers, deliberate treatment of specialized paths, an executable regression suite, and a structured review response.

Deterministic check	Weight	Pass	Fail
five package checks	0.40 total	rustls-test, util, examples, bogo, and bench each compile	Any named downstream package remains broken
workspace_callsite_migration	0.25	No executable old <code>`.read_tls(...)`</code> callsite remains	Two competing ownership models or hidden shim
review_response	0.15	Feedback, changes, verification, and risk are recorded	Missing or non-technical response
api_regression_suite	0.20	Real repository API integration tests pass	Compile-only migration or behavior regression

Stage 3 gate

``min_reward = 0.80`` permits one limited miss to reach hardening. The final verifier rechecks workspace, ownership, review persistence, and runtime behavior, so promotion does not imply a benchmark pass.

Stage 4: Fault hardening and cumulative closure

The final fresh context receives operational edge cases. It must repair fragmentation, close-notify, EOF, backpressure, discard, benchmark, rollback, and integration failures without restoring the old ownership model.

Deterministic group	Weight	What executes	Why it is final
Hidden fragmentation/close-notify	0.12	Verifier-owned integration target	Tests novel transport chunking and close behavior
Eight fixed fault tests	0.32 total	Backpressure, clean/unclean close, junk/data after close, EOF	Closes operational edge cases
Final workspace matrix	0.12	Five downstream packages compile	Rechecks Stage 3 repository integration
Final ownership contract	0.12	Independent consumer plus no old callsites/second buffer	Rechecks the load-bearing architecture
Benchmark target	0.06	Benchmark crate compiles	Protects performance tooling
Verifier rollback check	0.06	`git diff --check` plus core build	Independent merge safety
Agent rollback artifact	0.03	Executable script runs required checks without swallowing failure	Tests handoff/tool quality
Persisted review response	0.05	Stage 3 response remains valid	Closes the process loop
Three structured artifacts	0.12 total	Fault matrix, benchmark evidence, final report	Captures executable claims for review/training

Final gate

Every weighted final check and integrity must pass for `binary\_reward = 1`. No judge score can rescue a failure. A partial deterministic score is retained only as labeled training signal.

8. Deterministic Evaluation Design

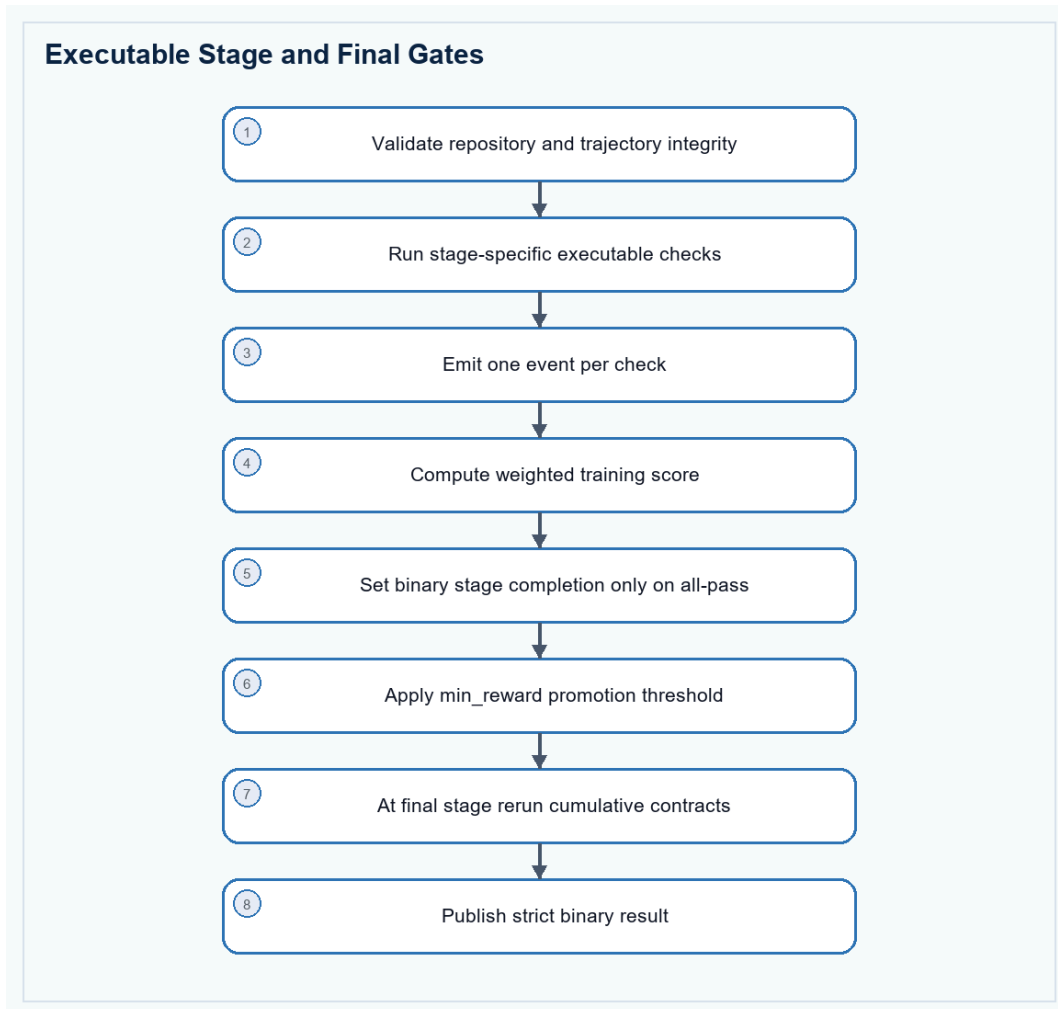


Figure 7. Deterministic checks create both hard stage gates and granular training events without patch matching.

Check family	Examples	Use as strict truth?	Design rule
Fail-to-pass	Original symptom plus hidden variants	Yes	Exercise behavior, not golden lines
Pass-to-pass	Existing focused suites and downstream packages	Yes	Run where prompt scope makes them fair
Property/metamorphic	Chunk sizes, ordering, equivalent inputs, retries	Yes when seeded and stable	Vary irrelevant shape while preserving expected behavior
Integration	Real consumers, services, schemas, protocol flows	Yes	Prefer system interfaces over internal symbol checks
Fault injection	EOF, close, backpressure, process/service failure	Yes when deterministic	Use fixed fault points and bounded clocks
Performance	Buildable benchmark or calibrated budget	Yes only when stable	Compare repeated Oracle baselines and allow noise
Artifact	Executable reproducer, rollback check, typed handoff	Yes when explicitly requested	Validate behavior/schema, not prose style
Source shape	No second buffer or obsolete public call shape	Only for load-bearing contract	Prompt must make architecture requirement clear
Integrity	No verifier/solution/history/network/reward access	Always	Invalid trial cannot be promoted

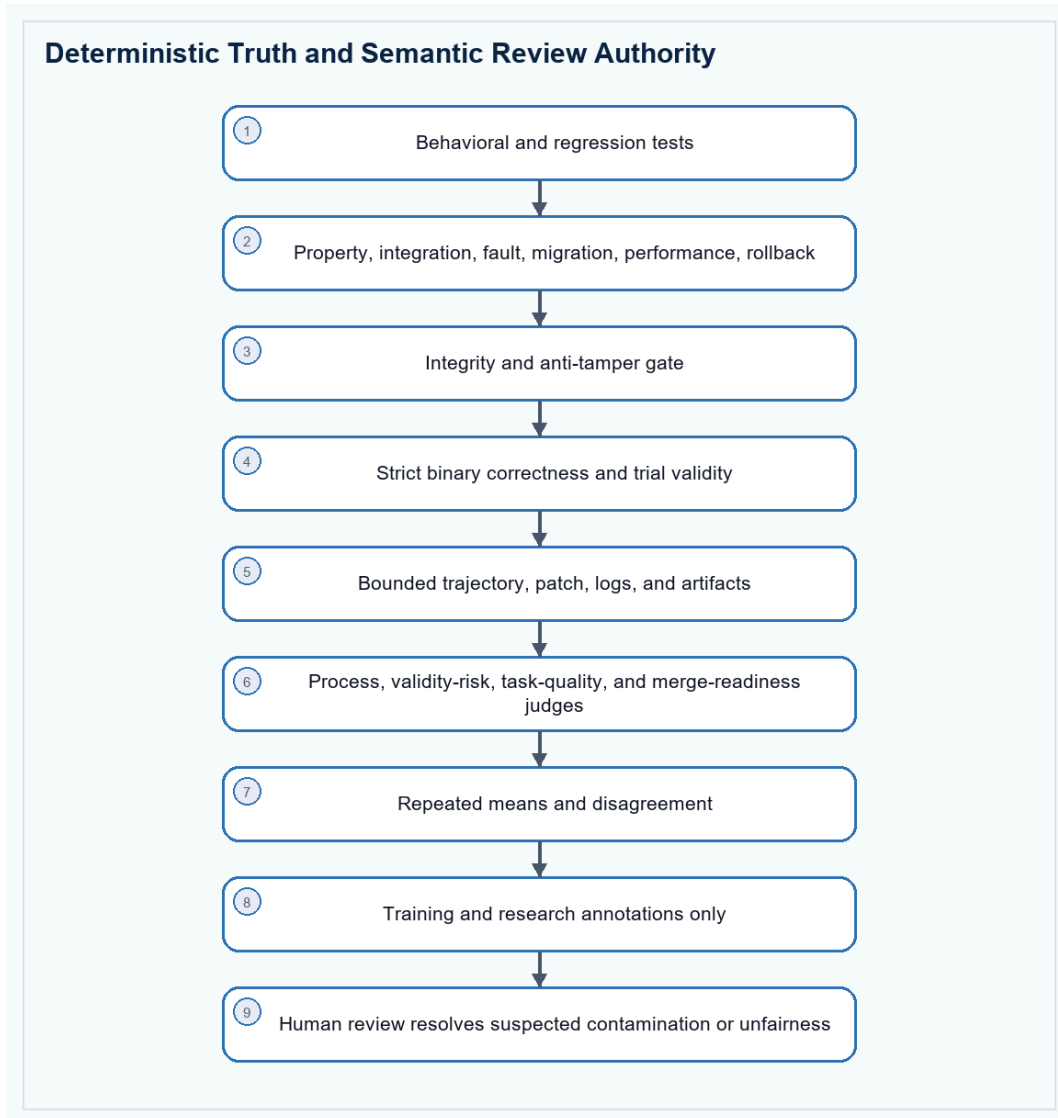


Figure 7A. Executable checks own correctness, integrity owns validity, judges own semantic annotations, and human review resolves ambiguous invalidation.

Control type	Examples	Can fail strict eval?	Can add training signal?
Deterministic behavior	Fail-to-pass, pass-to-pass, property, integration, fault, migration, performance, rollback	Yes	Yes, one named event per check
Deterministic process/integrity	Forbidden paths/commands, network, history, test tampering, symlink escape, reward spoofing	Yes; confirmed violation makes the trial invalid	Yes, but quarantine invalid trajectories
Structured artifact	Typed handoff, reproducer, migration proof, benchmark, rollback script	Only when explicitly requested and behaviorally validated	Yes
Process judge	Evidence use, diagnosis sequence, handoff use, self-correction	No	Yes
Validity judge	Possible leakage, shortcut, benchmark defect, infrastructure issue	No by itself	Yes; creates review flags
Task-quality judge	Prompt/check alignment, stage fairness, evidence sufficiency	No	Authoring and failure-label signal
Merge-readiness judge	Minimality, idioms, architecture, tests, operability	No; run only after deterministic final pass	Yes, preference and review signal

Deterministic process and integrity gate

```
def integrity_gate(testbed, trajectory):
    changed = tracked_and_untracked_paths(testbed)
    forbidden_paths = paths_matching(
        changed,
        [".git/**", "tests/**", "solution/**", "**/reward*.json"],
    )
    command_violations = audit_agent_commands(
        trajectory,
        forbidden=[
            "external URLs or source hosts",
            "curl/wget or runtime package installation",
            "git log/show/fetch/remote/reflog",
            "verifier, solution, Docker socket, or credential paths",
        ],
    )
    artifact_violations = reject_symlink_escape_or_spoofed_logs()
    passed = not (forbidden_paths or command_violations or artifact_violations)
    return passed, {
        "changed_files": changed,
        "forbidden_paths": forbidden_paths,
        "command_violations": command_violations,
        "artifact_violations": artifact_violations,
    }
```

- Treat integrity as a zero-weight validity gate, not a source of easy reward. A clean run may advance; a confirmed violation sets `valid\_trial = 0` and promotable reward to zero.
- Record all tracked and untracked files, relevant process commands, network attempts, protected-path access, verifier checksums, and artifact path resolution.
- Reject symlinks or hard links that escape the allowed output directory and never trust agent-written reward or test-result files.
- Distinguish suspicious from confirmed behavior. Patch similarity, good intuition, or fast completion alone does not prove cheating.
- Preserve raw evidence and use a human or deterministic confirmation before assigning a contamination label that removes a trial from evaluation or training.

Shared verifier entrypoint

```
#!/usr/bin/env bash
set -uo pipefail

stage="${1:?stage name required}"
RESULT_DIR="${RESULT_DIR:-/logs/verifier}"
mkdir -p "$RESULT_DIR"

python3 /tests/stage_check.py --stage "$stage" > "$RESULT_DIR/test_output.log" 2>&1

# Every path must leave a numeric result, even if a checker crashes.
if [ ! -f "$RESULT_DIR/reward.json" ]; then
    printf '%s\n' '{"reward":0,"binary_reward":0,"training_score":0,"valid_trial":0}' \
    > "$RESULT_DIR/reward.json"
fi

git -C /testbed diff --binary > "$RESULT_DIR/patch.diff" 2>/dev/null || true
python3 /tests/stage_judge_inputs.py || true

if [ "${RUN_LLM_JUDGES:-0}" = "1" ]; then
    TASK_STAGE="$stage" bash /tests/run_optional_judges.sh "$RESULT_DIR" || true
fi
exit 0
```

- Do not compare the candidate patch with the golden patch. The Oracle proves solvability; it is not the acceptance matcher.

- Write a numeric reward on every verifier path. A verifier crash must become an explicit invalid zero, not a missing result.
- Keep stage checks aligned with the stage prompt. A downstream package deferred to Stage 3 must not be a strict Stage 2 check.
- Seed near misses for narrow local fixes, duplicate compatibility paths, missing callers, broken features, prose-only artifacts, and fault cases.
- Require all final cumulative checks even if earlier partial promotion was allowed.

9. Non-Deterministic Judges That Run Correctly

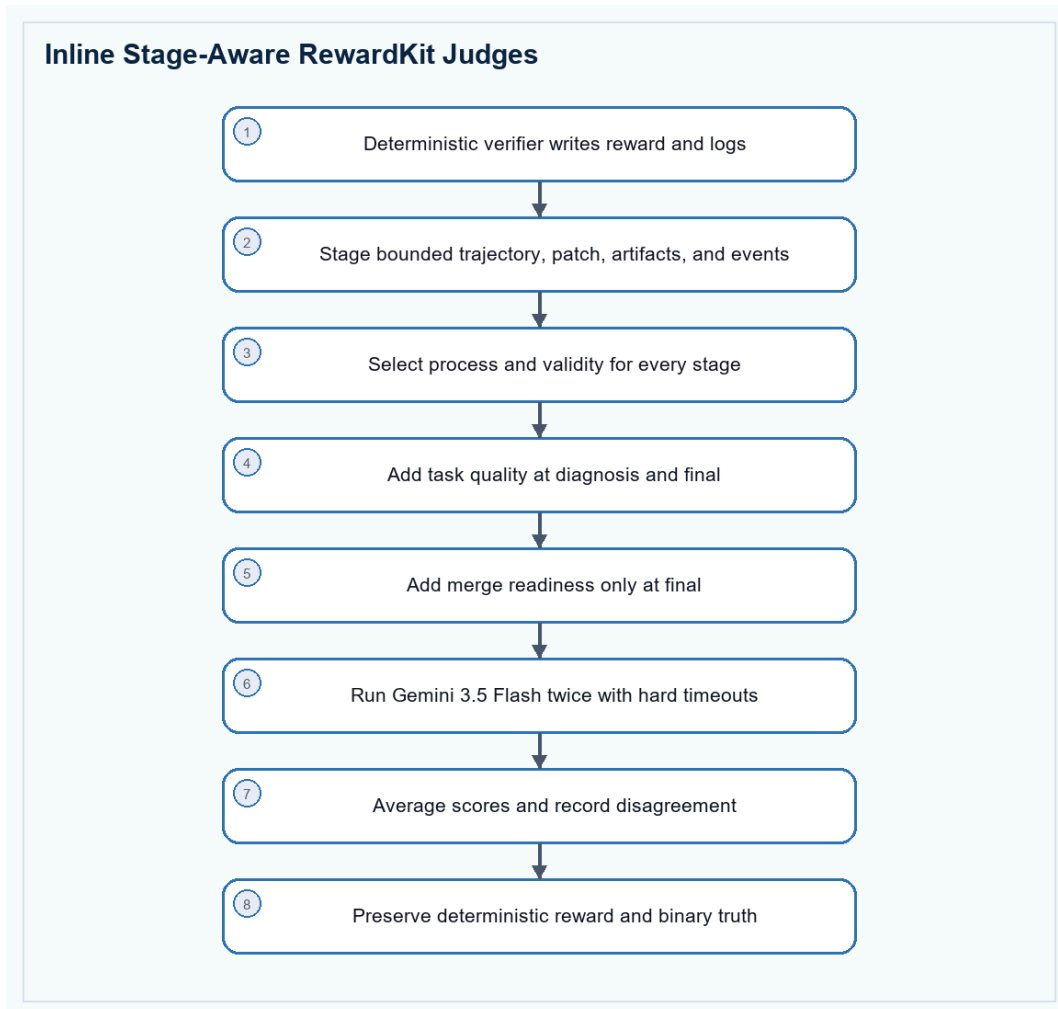


Figure 8. Judges run inside the verifier after deterministic checks and annotate the result without changing strict correctness.

Judge dimension	Stages	Question	Authority
process	All four	Did reasoning follow evidence, use handoffs, verify, and self-correct?	Research/training score only
validity	All four	Was there leakage, shortcutting, reward hacking, or an infrastructure/verifier issue?	Review flag; deterministic or human confirmation required
task_quality	Diagnosis and final	Did prompt, state, checks, and gates measure the intended work fairly?	Author audit only

Judge dimension	Stages	Question	Authority
merge_readiness	Final only	Is a deterministic-passing patch focused, idiomatic, coherent, and reviewable?	Zero unless final binary pass; cannot rescue failure

Judge criteria by stage

Stage	Process criteria	Validity and task-quality criteria	Merge-readiness criteria
1. Diagnose	Inspects relevant boundaries; creates a real reproducer; separates observation from hypothesis; leaves an actionable typed handoff	Logs are genuine; no solution/history/network access; strict requirements are disclosed and supported by the prompt	Not applicable
2. Core	Uses handoff; changes the load-bearing boundary before broad cleanup; runs focused tests; reports deferred integration honestly	No hidden compatibility path, canned output, test weakening, or claim beyond executable evidence	Not applicable
3. Review	Addresses newly revealed feedback; traces downstream contracts; verifies migrations and regressions; updates the handoff/review response	No selective filtering, concealed broken caller, pre-revealed feedback, or artifact tampering	Not applicable
4. Harden	Uses fault evidence; tests recovery and rollback; compares performance evidence; closes earlier gaps and verifies final state	No late shortcut, leaked hidden case, benchmark gaming, or prompt/final-verifier mismatch	Only after strict pass: focused diff, repository conventions, coherent architecture, adequate tests, operational safety, no patch bloat

Judge evidence contract

Give the judge only the current stage instruction, bounded trajectory, changed sources, raw deterministic outputs, typed artifacts, exact failure reasons, and prior persistent handoffs. Do not provide the golden patch, hidden tests, private provenance, unrestricted repository history, or credentials. Repeat the judge, record per-criterion outputs and disagreement, and keep its scores outside `binary\_reward`.

```
[judge]
judge = "gemini/gemini-3.5-flash"
prompt_template = "../judge_prompt.md"
files = ["logs/verifier/judge_inputs"]

[[criterion]]
name = "stage_specific_reasoning"
bucket = "process"
description = '''Review only bounded trajectory, patch, artifacts, and test
evidence. Reward evidence-backed reasoning for the current stage; do not infer
correctness in place of executable tests.'''
type = "binary"
weight = 1.0
```

Validated judge runtime

The verifier image contains Python 3.12, pytest 8.3.5, and harbor-rewardkit 0.1.7. Six contract tests pass in that image, including all stage-selection cases and preservation of deterministic rewards. The same RewardKit and `gemini/gemini-3.5-flash` path completed two real calls in the latest Tier 1 run. A Tier 2 provider call still requires `GEMINI\_API\_KEY` at run time; unavailable or rejected credentials produce `judge\_status.json` and never change binary truth.

Judge failure handling

- No key: mark the judge unavailable and retain the deterministic result.
- Network/DNS failure: classify provider unreachable and keep logs under `judge\_runs/`.
- 401/403: report a rejected key without printing or storing the key.
- 404/model unavailable: report model configuration failure.
- Timeout: terminate each call after the configured bound and record uncertainty.
- Judge disagreement: export the spread; never treat one stochastic score as hard truth.
- Deterministic failure reason present: require the judge to cite it and prohibit unsupported alternative explanations.
- Prompt/check mismatch: mark task-quality risk and route the trial for author review rather than calling it a clean model failure.

10. Reward Shape and Promotion Logic

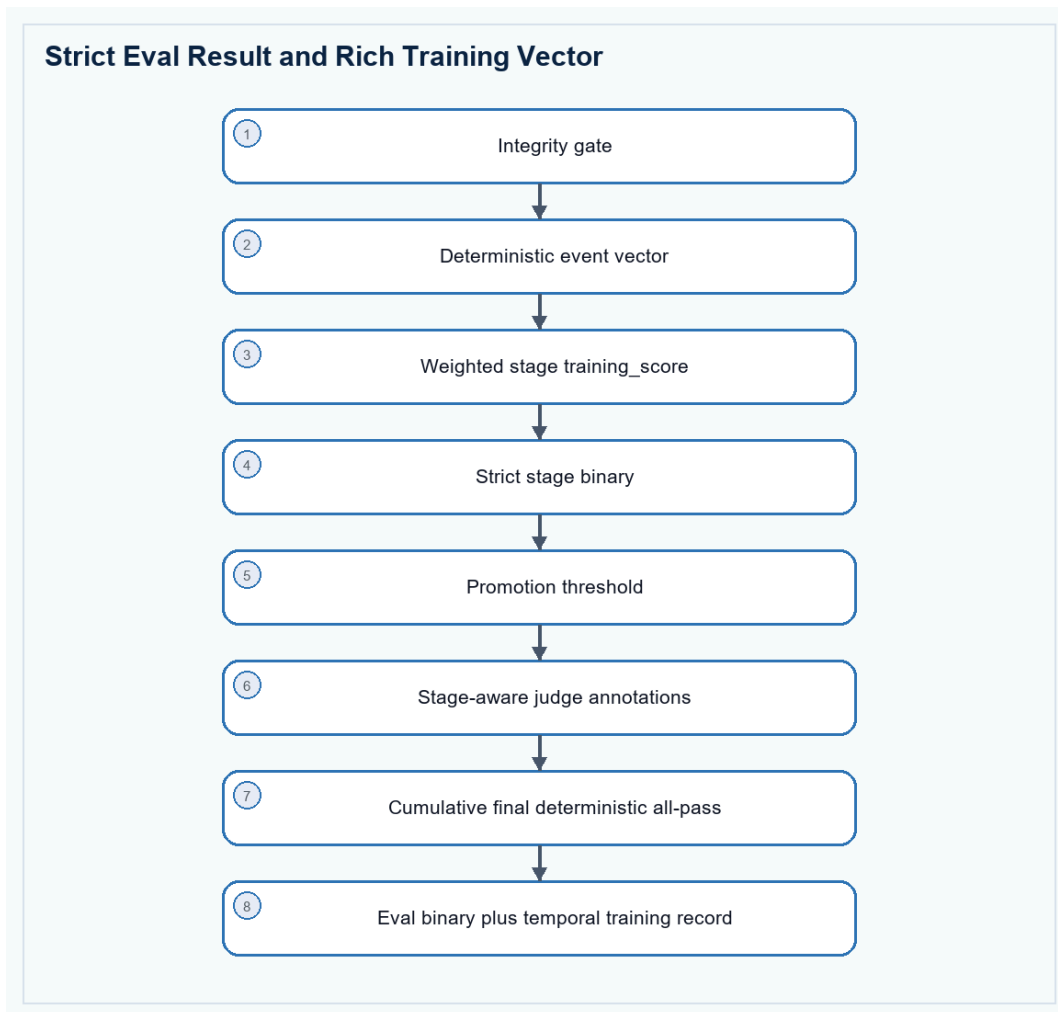


Figure 9. Early weighted rewards support recovery and learning; only cumulative all-pass yields a benchmark solve.

Output	Meaning	Used for	Hard rule
reward	1.0 on strict stage pass; otherwise weighted deterministic progress	Harbor 'min_reward' promotion	Integrity failure forces promotable reward to zero
binary_reward	Strict all-pass for the current/final stage	Benchmark success	Judge cannot change it
training_score	Weighted deterministic check completion	Temporal credit and near-miss ranking	Never call a partial score solved
valid_trial	Repository and process boundary remained intact	Eligibility and quarantine	Zero blocks promotion/training use
stage_complete	All deterministic stage checks passed	Stage analytics	Separate from threshold-based advancement
process_judge	Evidence-backed engineering process	Training/research	No binary authority
merge_ready_score	Final code quality	Preference data and review research	Zero unless final deterministic pass
research_score	0.70 training + 0.20 process + 0.10 merge readiness	Model analysis only	Not an eval pass condition

General weighting rules

- Let deterministic stage weights sum to 1.0, but designate load-bearing behavior and integrity as mandatory gates where appropriate. A judge score never fills a missing deterministic fraction.
- Give most weight to executable behavior, regression safety, integration, data correctness, faults, migration, performance, or recovery. Do not make file creation or document completion a large reward source.
- An artifact earns deterministic credit only when the verifier executes or cross-checks it. For example, rerun the reproducer, benchmark, migration check, rollback script, or structured handoff contract.
- Keep integrity at zero reward weight and use it as eligibility. Models should not earn points merely for avoiding cheating.
- Keep process, validity, task-quality, and merge-readiness judge scores as separate dimensions with repeated results and uncertainty. Do not blend them into the benchmark pass/fail value.
- Calibrate promotion thresholds with authored partial patches. Every score that can advance must represent enough durable state for the next stage to be meaningful.

Why partial promotion is useful

A model that scores 0.80 in Stage 2 may still expose whether later review and hardening can recover the missing 0.20. This produces useful recovery trajectories. The final cumulative all-pass requirement prevents that experimental flexibility from inflating the benchmark solve rate.

11. How to Run and Read Results

```
# Use Harbor 0.8 or newer; multi-step task layout is not handled correctly by
# older Harbor 0.3 installations that may also exist on the same WSL machine.
harbor --version

# Controls: use a fresh job name each time.
harbor run -p ./<tier2-task> \
  -a oracle --jobs-dir /root/harbor-jobs \
  --job-name <task>-oracle-v1

harbor run -p ./<tier2-task> \
  -a nop --jobs-dir /root/harbor-jobs \
  --job-name <task>-nop-v1

# One Terminus job with judges executed inside every stage verifier.
export GEMINI_API_KEY='your-google-ai-studio-key'
export JUDGE_REPEATS=2
bash ./<tier2-task>/tools/run_inline_judged_terminus.sh \
  <task>-terminus-judged-v1
```

Worked rustls control commands

```
# Earlier v9 worked-sample controls retained from the rustls package.
harbor run -p ./rustls-external-input-tier2 \
  -a oracle --jobs-dir /root/harbor-jobs \
  --job-name rustls-tier2-v9-oracle

harbor run -p ./rustls-external-input-tier2 \
  -a nop --jobs-dir /root/harbor-jobs \
  --job-name rustls-tier2-v9-nop

# Current v11 deterministic regression after the disclosed-contract fix.
harbor run -p ./rustls-external-input-tier2 \
  -a oracle --jobs-dir /root/harbor-jobs \
  --job-name rustls-tier2-oracle-contract-fix-v1 \
  --ve RUN_LLM_JUDGES=0

export GEMINI_API_KEY='your-google-ai-studio-key'
bash ./rustls-external-input-tier2/tools/run_inline_judged_terminus.sh \
  rustls-tier2-terminus-judged-v3
```

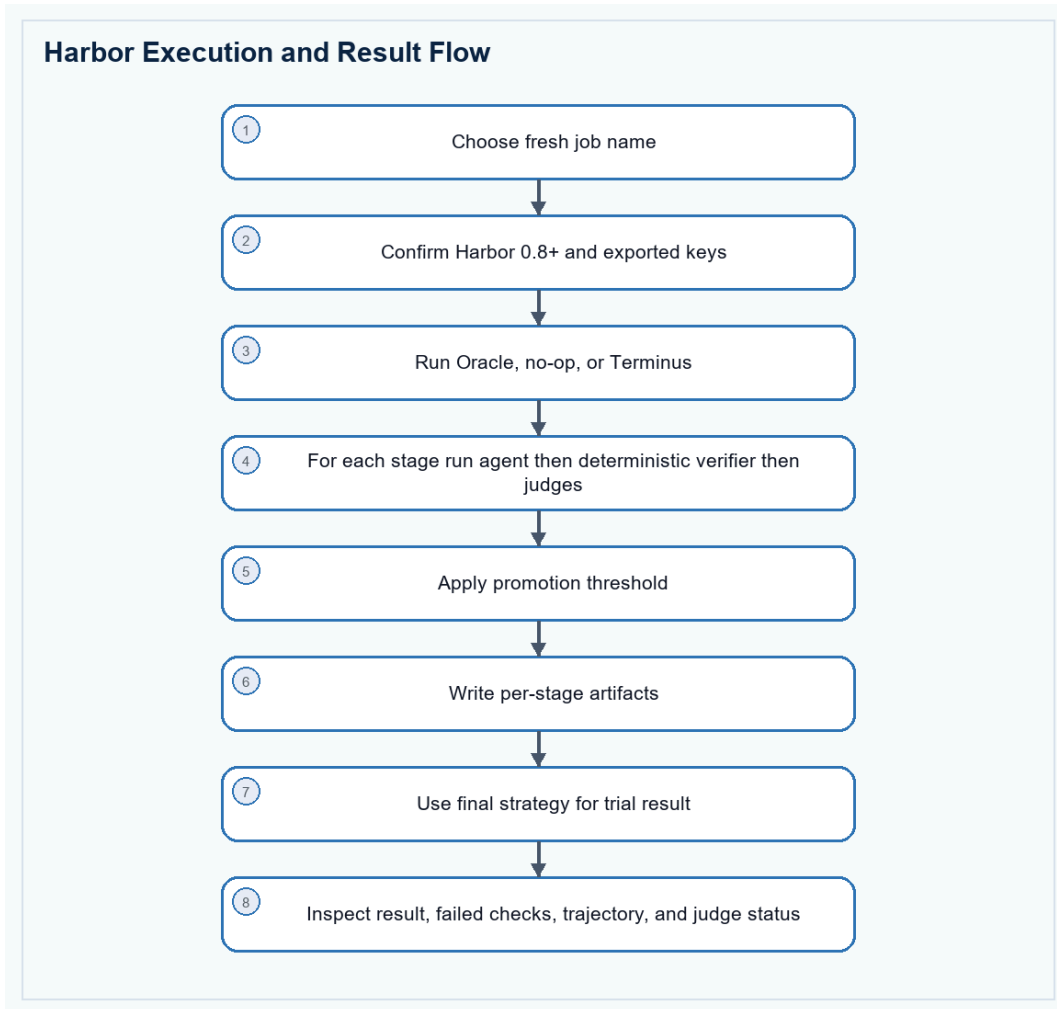


Figure 10. One Harbor command runs all stages and inline judges, then stores isolated evidence for every stage.

```

/root/harbor-jobs/<job-name>/
|-- result.json           # trial-level final strategy output
|-- config.json           # actual agent, model, verifier env
+-- <task-name>__<trial-id>/
  |-- result.json
  +-- steps/
    |-- 01-diagnose/
    |-- agent/trajectory.json
    |-- artifacts/
    +-- verifier/
      |-- reward.json
      |-- reward-details.json
      |-- reward-events.json
      |-- judge_status.json
      |-- judge_summary.json      # present when judge succeeds
      |-- judge_runs/repeat-*/
      |-- training_export_manifest.json
      +-- test_output.log
    |-- 02-core-migration/...
    |-- 03-review-integration/...
    +-- 04-hardening-final/...
  
```

Read results in this order

- Job `result.json`: confirm actual agent, model, errors, final `binary\_reward`, and which stage ended the trial.
- Per-stage `reward.json`: distinguish strict completion from partial training progress.

- ``reward-details.json``: read the exact failed deterministic checks and their evidence.
- ``test_output.log``: identify compiler, test, timeout, environment, or script failure.
- ``judge_status.json``: confirm provider availability, selected dimensions, repeats, means, and disagreement.
- Trajectory and patch: decide whether failure was a model miss, unfair prompt/check mismatch, contamination, or infrastructure problem.

Validated v9 controls

The original v9 controls are retained below, followed by the current v11 disclosed-contract regression results.

Run	Observed result	Interpretation
Oracle v9	All four stages <code>`binary_reward = 1`</code> ; final result 1.0; about 4m20s in the cached local image	The revised prompts, stage gates, Oracle scripts, and cumulative final verifier were jointly solvable
No-op	Stopped at Stage 1; <code>`binary_reward = 0`</code> ; <code>`training_score = reward = 0.15`</code> ; <code>`valid_trial = 1`</code>	Untouched code cannot advance; the small partial value is only base-state observation
Reward contract v9	Six pytest cases passed in the earlier verifier image	Required files and all stage-aware judge dimension selections were deterministic
Judge provider	Gemini 3.5 Flash model ID is current and the equivalent Tier 1 RewardKit path completed two calls	Tier 2 is correctly wired; each run still needs an exported key and verifier network
Oracle v11	Fresh Harbor run <code>`rustls-tier2-oracle-contract-fix-v1`</code> : all four stages and final binary reward 1.0; 16m29s including a cold image build	The disclosed Stage 1 reproducer contract promotes and unchanged later stages still close cumulatively
Reward contract v11	Ten Linux package tests pass, including exact missing-requirement reporting and stage-aware judge selection	The verifier names failures for the judge instead of inviting speculative explanations

12. How to Formulate Similar Tier 2 Tasks

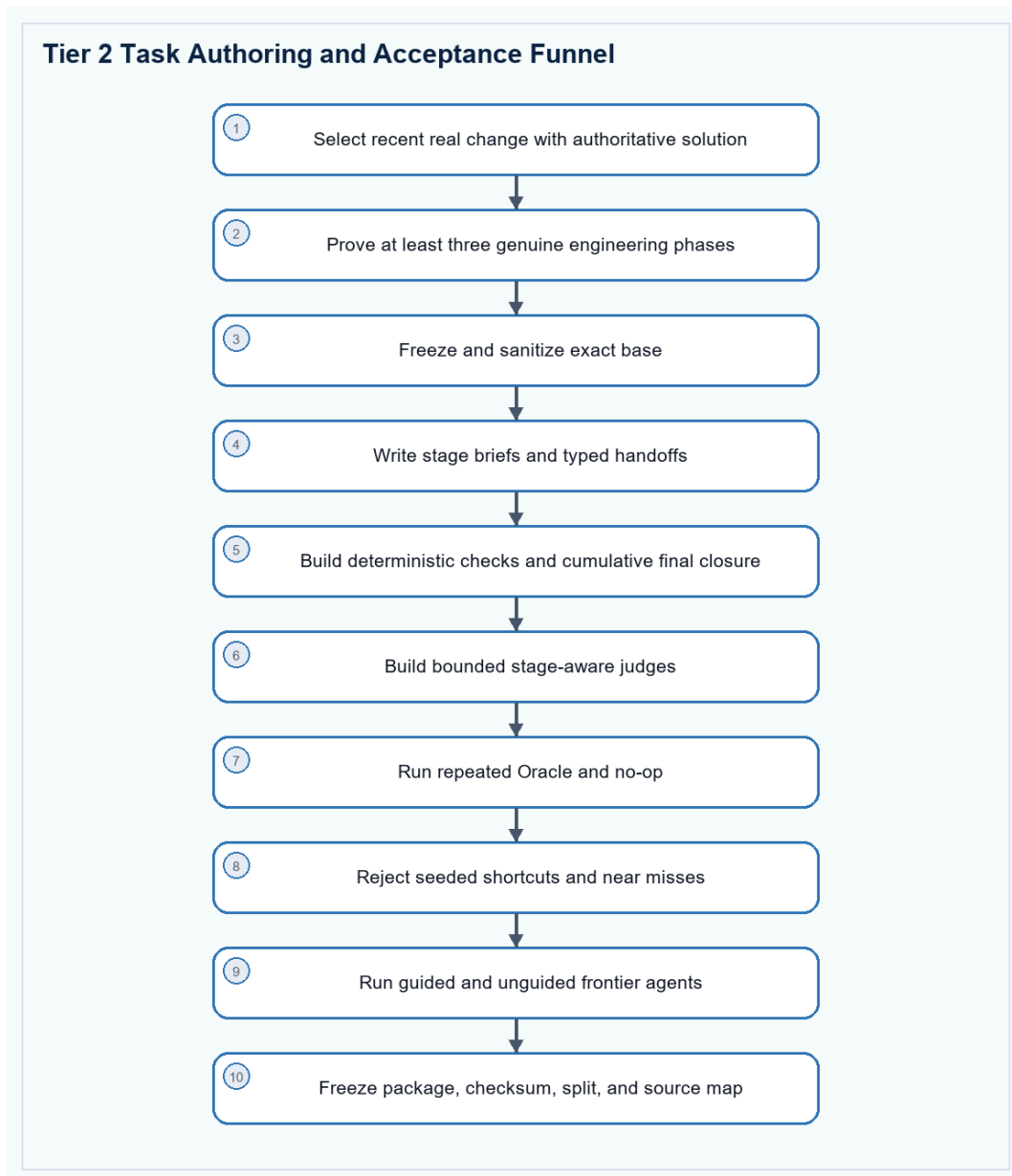


Figure 11. A task becomes benchmark material only after scope, fairness, solvability, shortcut resistance, and frontier headroom are demonstrated.

Choose work that is genuinely stateful

- Prefer migrations, concurrency fixes, distributed state, protocol changes, storage/transaction behavior, compatibility transitions, performance work, or fault recovery.
- Require at least three distinct phases that a real engineer would recognize, not arbitrary checkpoints inserted into one local edit.
- Make later evidence causally depend on earlier work: CI failures, review comments, production traces, load results, migration state, or fault tests.

- Keep the complete repository and real service topology. Do not extract the target function into a toy project.
- Use a recent merged PR or maintainer patch for provenance and Oracle construction, then create novel hidden variants so PR recall alone is insufficient.

Language and repository selection

Language	Strong domains	Useful Tier 2 signal	Authoring caution
Rust	Networking, compilers, storage, async systems, security	Ownership, feature matrices, workspace callers, fault behavior	Pin toolchain and prebuild expensive targets
C/C++	Databases, browsers, media, compilers, systems	ABI, memory, concurrency, performance, rollback	Use deterministic sanitizers and bounded workloads
Go	Cloud control planes, networking, operators	Fast broad tests, races, service state, fault injection	Add real state and contracts; avoid tiny local fixes
Java/Kotlin	Large services, transactions, build systems	Multi-module integration, compatibility, migrations	Pin JDK/build caches and control external services
Elixir	Concurrent services, queues, data pipelines	Runtime investigation, supervision, race and recovery behavior	Seed state deterministically and bound timing
Python/TypeScript	Data platforms, web products, orchestration	Cross-service workflows, schemas, UI/API contracts	Difficulty must come from the system, not language verbosity

Split the stages by engineering evidence

- Stage 1 should produce an executable reproducer and a structured technical decision, not only a document.
- Stage 2 should implement one coherent load-bearing boundary while explicitly deferring broader integration.
- Stage 3 should reveal new reviewer/CI evidence and require repository-wide integration and response.
- Stage 4 should exercise faults, rollback, performance, migrations, or operational recovery and close all earlier gaps.
- Every stage must leave evidence useful to the next context and a reward event vector useful to training.

Acceptance calibration

Control	Required outcome	Failure action
Oracle repeated clean runs	Every stage strict-passes	Fix environment, prompt, verifier, or Oracle before model trials
No-op repeated runs	Stops at the earliest meaningful gate	Strengthen checks or remove unearned artifact credit
Seeded near misses	Each fails the intended named gate	Add discriminating hidden/property/integration checks
Hidden mutation rejection	At least 90% of authored shortcuts rejected	Do not accept task yet
Guided frontier	At least one strong model proves practical solvability	Investigate ambiguity or environment blockers
Unguided frontier	Roughly 5-20% strict solve rate	Increase stateful difficulty or remove arbitrary strictness
Judge repeatability	Oracle/no-op semantic outcomes stable across repeats	Revise rubric, evidence bounds, or judge role

Seed the failures before testing frontier models

Authored candidate	What it tests	Expected rejection
No-op and prompt-shaped artifact only	Unearned base credit and prose/file-completion gaming	Stops at diagnosis; cannot promote

Authored candidate	What it tests	Expected rejection
Golden behavior with one hidden variant missing	Verifier discrimination beyond the visible example	Fails property, metamorphic, or hidden behavior event
Local fix with downstream caller broken	Cross-component and pass-to-pass coverage	Fails integration or regression event
Compatibility shim retaining obsolete path	Architecture and migration completion	Fails external contract, source/runtime invariant, or final closure
Hardcoded marker or fabricated log	Evidence spoofing resistance	Verifier rerun fails and validity flag is raised
Project test deleted, skipped, filtered, or weakened	Test tampering controls	Integrity/regression gate invalidates the run
Source PR retrieved or verifier path accessed	Network/history/protected-path controls	Confirmed violation sets 'valid_trial = 0' and quarantines the trace
Correct but bloated strict-passing patch	Judge separation	Binary pass remains; merge-readiness score falls
Fair alternative architecture	Patch-independence	Passes when observable contracts and disclosed invariants hold

13. Evaluation-to-Training Design

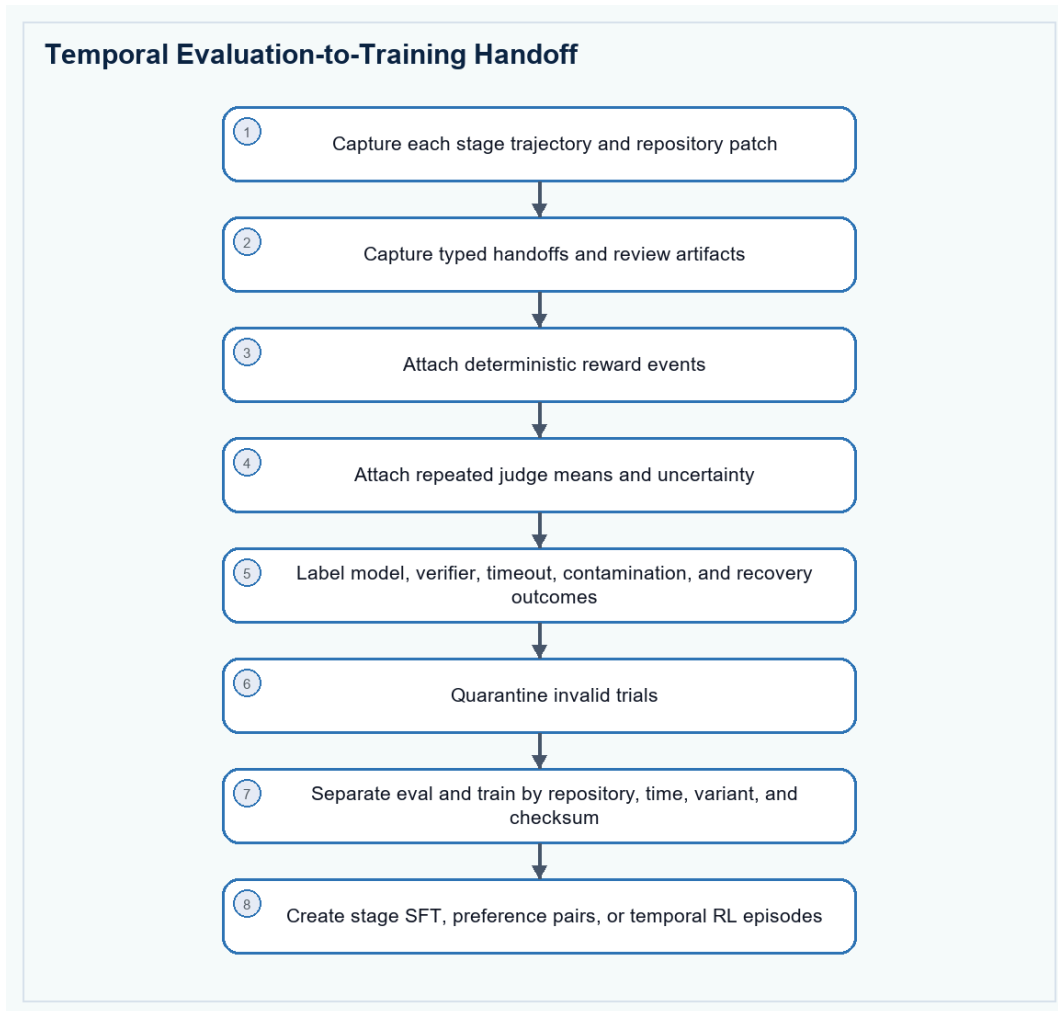


Figure 12. Tier 2 exports not only a final patch but the sequence of decisions, failures, feedback responses, and recoveries that produced it.

```
{
  "schema": "repo-engineering-training-v2",
  "task_family": "<task-family>",
  "stage": "<diagnose|core|review|harden>",
  "training_candidate": true,
  "eligible_for_training": false,
  "split_policy": "never train on an active evaluation instance",
  "artifacts": [
    "trajectory.json", "patch.diff", "changed_files.txt",
    "reward-details.json", "reward-events.json", "test_output.log",
    "agent_output/", "judge_inputs/", "judge_summary.json"
  ],
  "failure_labels": ["<failed deterministic gates>"],
  "contamination_label": "<clean|suspected|invalid>"
}
```

Training use	Tier 2 record	Why it is useful	Safety rule
Stage SFT	Good diagnosis, implementation, review response, or hardening segment	Teaches phase-specific engineering behavior	Exclude active eval task identity and hidden verifier content
Temporal credit	Per-check reward events across ordered stages	Shows where durable work succeeded or failed	Do not collapse partial reward into a solved label
Recovery preference	Partial stage followed by successful versus failed recovery	Trains use of feedback and persistent state	Compare only equivalent valid starting states
Merge-readiness preference	Two strict-passing final patches with quality scores	Teaches minimality, conventions, and design taste	Never prefer stylish broken code over correct code
Failure classification	Model miss, prompt ambiguity, verifier defect, environment, timeout, contamination	Prevents training on mislabeled failures	Human or deterministic confirmation for invalidation

14. Worked rustls Sample: What Is Real and What Is Benchmark Design

The sample uses the complete `rustls/rustls` repository at base commit `68771ca492c91b1084ea6d51354385aef62b5ea4`. Its author provenance is merged rustls PR 2940, `Externalize input buffers`, merged July 7, 2026. The PR proves that the core engineering direction is real. The benchmark adds independent staged prompts, typed handoffs, hidden variants, review feedback, cumulative fault checks, integrity controls, and training exports.

Element	Taken from real engineering work	Added by benchmark design
Repository	Complete rustls worktree and real build graph	Sanitized base without origin, future history, solutions, or verifier files
Core change	External caller-owned TLS input-buffer migration	Fresh staged contexts and explicit typed handoff
Integration	Real client, server, acceptor, utilities, examples, Bogo, and benchmarks	Stage 3 simulated review/CI reveal and promotion threshold
Hardening	Real close, EOF, backpressure, fragmentation, and performance concerns	Verifier-owned hidden variants, rollback evidence, and cumulative final reward
Scoring	Repository tests and build behavior	Strict binary, temporal event vector, process/validity/task-quality/merge judges

Contamination rule

The source PR is author provenance, not agent-facing context. The solving image contains no origin, PR number, future commit, golden patch, or verifier. Hidden cases and staged feedback must make memorized public patch recall insufficient for success.

15. Final Author Checklist

- Use Harbor 0.8 or newer and the official multi-step folder layout; do not add single-step root prompt/solution files.
- Declare schema 1.3, metadata.benchmark_type, ordered steps, promotion thresholds, and final reward strategy.
- Use a complete sanitized repository at an exact pre-change commit with no origin, reflog, future refs, or source-change clues.
- Sanitize by exporting an exact tree from a disposable clone and initializing one synthetic commit; never rewrite or delete history in the maintainer checkout.
- Assert one synthetic commit, no remotes/tags/remote refs, no source reflog, no source PR identifier, and no future-solution clue in agent-visible files.
- Pin language and verifier dependencies; verify Python 3.12, pytest, RewardKit, and the language toolchain inside the image.
- Write realistic stage briefs with explicit current scope, preserved behavior, deferred work, and typed durable artifacts.
- Align every strict stage check with work that stage has actually requested.
- Cover fail-to-pass, pass-to-pass, properties, integration, faults, artifacts, and integrity as appropriate.
- At every stage run deterministic behavior checks, deterministic process/integrity checks, typed artifact validation, process/validity judges, and a documented promotion decision.
- Block or allowlist agent network; protect solution, verifier, credentials, Docker socket, host mounts, sibling jobs, and reward outputs.
- Audit tracked/untracked paths, commands, history access, network attempts, test weakening, symlink escape, fake logs, and reward spoofing.
- Disclose every strict command, path, schema, and artifact contract in the prompt unless it is enforced behaviorally by the environment.
- Return exact machine-readable deterministic failure reasons and pass them to judges; prohibit speculative alternative causes without artifact evidence.
- Use partial promotion only to study recovery; require cumulative final deterministic all-pass for benchmark success.
- Run process and validity judges at every stage, task quality at diagnosis/final, and merge readiness only after final deterministic pass.
- Keep API keys in verifier environment variables; bound judge model, evidence, repeats, and timeouts.
- Require repeated clean Oracle pass, repeated no-op stop, seeded near-miss rejection, guided solvability, and frontier calibration.
- Export trajectories, stage snapshots, handoffs, patches, reward events, tests, judges, failure labels, and contamination labels.
- Keep training and evaluation separate by repository, time window, task variant, hidden cases, and checksum.
- Quarantine confirmed contaminated or invalid trials; do not label fast work or golden-patch similarity as cheating without stronger evidence.

16. Source Map

Sources were rechecked on July 14, 2026. Current benchmark facts are dated because leaderboards, task counts, model IDs, and Harbor behavior can change. Public source-change identity belongs in author provenance or this worked example, not in the protected agent prompt.

Tier 2 Stateful Repo Engineering Task Guide

Source	URL	Used for	Design decision
SWE-bench	swebench.com	Containerized issue-to-patch repository evaluation	Retain executable behavior and pass-to-pass safety
SWE-Bench Pro	labs.scale.com	Full professional repositories and task containers	Use real repositories, but avoid over-specified patch recipes
Senior SWE-Bench overview	senior-swe-bench.snorkel.ai	Current task scope, public/private counts, stacks, and leaderboard framing	Compare against its natural briefs and senior-quality bar
How Senior SWE-Bench works	senior-swe-bench.snorkel.ai	Behavioral verifiers, validation agent, taste judge, and quality control	Do not claim novelty for capabilities already present there
Senior SWE-Bench performance	senior-swe-bench.snorkel.ai	Correctness, taste, trajectory analysis, and error modes	Keep process labels useful without replacing runtime truth
Senior SWE-Bench v2026.06 tasks	github.com	Concrete public Harbor packages	The public tasks use the conventional single-task layout
Harbor task structure	harborframework.com	Task, verifier, artifacts, network, and protected files	Keep one reproducible package and a numeric reward contract
Harbor multi-step tasks	harborframework.com	Shared environment, steps layout, min_reward, and final strategy	Use stage directories, persistent state, and explicit early gates
Gemini 3.5 Flash	ai.google.dev	Stable model ID and long-horizon/coding capability	Use gemini/gemini-3.5-flash through RewardKit/LiteLLM
rustls PR 2940	github.com	Real merged external-input-buffer migration and performance evidence	Use only as private author provenance and a worked example